

GOFORGE

AI-Aware Backend Code Generation Platform

Product Specification

Version 4.0 · April 2026

CONFIDENTIAL

1. Executive Summary

GoForge is a deterministic, AI-aware backend code generation platform. It introduces a compact Domain Specific Language (DSL) in YAML that fully describes a backend service. From this single spec, GoForge generates 100% of structural code — handlers, repositories, service skeletons, migrations, event producers/consumers, external clients, jobs, auth, and observability wiring.

No AI is used inside the tool. All generation is deterministic and template-driven. Business logic is written by developers in hook files that the tool never touches.

Core Problem

AI coding agents spend the majority of their tokens writing structural boilerplate — handlers, repository functions, route wiring, migration SQL, error types, mock structs, and DI wiring. This code is 100% predictable from requirements yet the AI rewrites it from scratch every time.

GoForge Solution

Developer writes a compact YAML spec (~30–80 tokens per resource). GoForge generates all structural code deterministically. AI only writes business logic inside hook functions. Token usage drops by an estimated 75–85% for typical service development.

Key Principles

- Deterministic — identical spec always produces identical code. No AI in the tool.
- Immutable generated code — generated files are read-only. Humans never edit them.
- Hook-based extension — all custom logic injected via generated hook interfaces.
- Three-level escape hatch — hooks, custom endpoints, and full extensions for anything the DSL does not cover.
- Multi-language — Go (v1), Java Spring Boot (v2). Same spec, different output.
- Update-safe — regeneration never touches developer-owned files.

2. Problem & Opportunity

2.1 The Token Waste Problem

A typical AI-generated CRUD endpoint for a single resource involves the following files and approximate line counts:

File	Lines generated (approx.)
Migration SQL	15–25 lines — fully predictable from field list
Model / struct	20–40 lines — direct mapping of fields
Repository layer	60–120 lines — standard CRUD + query functions
Service layer	80–150 lines — mostly structural, ~20% real logic
Handler / controller	60–100 lines — fully predictable HTTP wiring
Route registration	10–20 lines — fully predictable
Error types	15–30 lines — fully predictable from rules
Mock structs	40–80 lines — mechanical interface implementation
DI wiring	20–40 lines — fully predictable
Total per resource	~320–600 lines, ~85% predictable boilerplate

2.2 Target Users

- Backend engineers using AI coding agents (Claude Code, Cursor, Copilot) to build services
- Teams building microservices who want consistency without framework lock-in
- Startups who want production-grade structure from day one without the setup cost
- Platform/infra teams standardising service generation across an org

2.3 Success Metrics

Metric	Target
AI token reduction	≥ 75% vs writing from scratch
Time to first running endpoint	< 5 minutes from empty spec

Regeneration safety	0 developer file overwrites ever
DSL expressiveness	Cover 80% of backend patterns without escape hatch
Generated code quality	Passes golangci-lint / SpotBugs with zero warnings

3. DSL Full Schema Reference

This section is the authoritative specification of every keyword in `goforge.yaml`. The tool validates all files against this schema before generating any code. Every key, its type, allowed values, and defaults are documented here.

Schema Notation

type: str | int | bool | enum[...] | list<T> | map. Required fields are marked (req). Optional fields show their default in parentheses. Inline comments explain what the tool generates from each field.

3.1 Top-Level Structure

The root of every `goforge.yaml` file. Defines the identity and target of the project.

```
version:      1                # (req) DSL schema version — always 1 for now
project:      orders-service   # (req) str — used as Go module name / Java
artifact ID
lang:         go               # (req) enum[go, java]
framework:    gin             # (req) enum[gin, echo, fiber] for go |
enum[spring] for java
db:           postgres        # (req) enum[postgres, mysql, mongo]
config:       [...]           # (req) list<ConfigVar> — see 3.2
resources:    [...]           # (req) list<Resource> — see 3.3
types:        [...]           # (opt) list<CustomType> — see 3.4
transactions: [...]           # (opt) list<Transaction> — see 3.5
auth:         {...}           # (opt) Auth block — see 3.6
messaging:    {...}           # (opt) Messaging block — see 3.7
externals:    [...]           # (opt) list<External> — see 3.8
jobs:         [...]           # (opt) list<Job> — see 3.9
cache:        {...}           # (opt) Cache block — see 3.10
storage:      {...}           # (opt) Storage block — see 3.11
observability: {...}          # (opt) Observability — see 3.12
middleware:   [...]           # (opt) list<str> — see 3.13
extensions:   [...]           # (opt) list<Extension> — see 3.14
overrides:    {...}           # (opt) Overrides block — see 3.15
```

3.2 Config Block

Declares all environment variables the service needs. The tool generates a typed `Config` struct and startup validation — the service will refuse to start if any required variable is missing or fails its constraint.

```
config:
  - name:      DATABASE_URL    # (req) str — env var name
    type:      str             # (req) enum[str, int, bool, enum]
    required:  true            # (opt) bool — default: false
    default:   8080            # (opt) any — used if var absent
    min_length: 32            # (opt) int — validated at startup (str only)
```

```

min:          1                # (opt) int - validated at startup (int only)
max:          65535            # (opt) int - validated at startup (int only)
values:       [debug,info,warn] # (opt) list<str> - for enum type
description:  "Primary DB URL"  # (opt) str - used in .env.example comments

```

Tool generates: typed Config struct, startup validator, .env.example with all keys and descriptions.

3.3 Resource Block

A resource maps to one domain entity, one DB table, and one full stack of generated files. It is the primary unit of the DSL.

```

resources:
  - name:      User              # (req) str - PascalCase, used for all type
  names
    table:     users             # (opt) str - default: snake_case plural of name
    fields:    [...]            # (req) list<Field> - see 3.3.1
    relations: [...]            # (opt) list<Relation> - see 3.3.2
    endpoints: [...]            # (opt) list<Endpoint> - see 3.3.3
    queries:   [...]            # (opt) list<Query> - see 3.3.4
    rules:     {...}            # (opt) Rules block - see 3.3.5
    states:    {...}            # (opt) StateMachine - see 3.3.6
    hooks:     {...}            # (opt) HookDecl block - see 3.3.7
    errors:    [...]            # (opt) list<str> - domain error names

```

3.3.1 Field Definition

All field properties use structured YAML — no terse string codes. This ensures unambiguous parsing and enables IDE schema validation.

```

fields:
  - name:      email            # (req) str - camelCase field name
    type:      str              # (req) enum[str,int,decimal,bool,date,
                                # timestamp,json,uuid,enum]
    required:  true             # (opt) bool - NOT NULL in DB + required
  validator
    unique:    true             # (opt) bool - UNIQUE INDEX + uniqueness check in
  service
    nullable:  true             # (opt) bool - NULL in DB, pointer type in Go
    private:   true             # (opt) bool - excluded from all API responses
    default:   user             # (opt) any - DB DEFAULT + struct zero value
    values:    [admin, user]    # (opt) list<str> - required when type: enum
    rules:     # (opt) list<Rule> - parameterised validators
      - type:  email            # no-param rule - string enum value
      - type:  min               # parameterised rule - structured map
        value: 2
      - type:  max
        value: 50
      - type:  min_length
        value: 8
      - type:  max_length

```

```

    value: 255
  - type: pattern          # regex validator
    value: "[a-zA-Z0-9_]+$"
  - type: url
  - type: phone            # E.164 format
  - type: alpha
  - type: alphanum
  added:    "v1.2"         # (opt) str - version field was added (docs only)
  deprecated: "v1.3"      # (opt) str - version field deprecated (docs only)
  removed:   "v1.4"       # (opt) str - field removed, no codegen for this
version

```

Field Type Mapping

DSL type	Go type	Java type	DB column type
str	string	String	VARCHAR(255)
int	int64	Long	BIGINT
decimal	float64	BigDecimal	NUMERIC(10,2)
bool	bool	Boolean	BOOLEAN
date	time.Time	LocalDate	DATE
timestamp	time.Time	Instant	TIMESTAMP TZ
json	json.RawMessage	JsonNode	JSONB
uuid	uuid.UUID	UUID	UUID
enum	custom type	enum class	VARCHAR + CHECK

Validation Rule Types (used in rules list)

Rule type	DB effect	Service layer effect
email	No DB effect	RFC 5322 email format validator
url	No DB effect	URL format validator
phone	No DB effect	E.164 phone format validator
alpha	No DB effect	Letters only validator
alphanum	No DB effect	Letters and digits only
min (int)	CHECK (col >= value)	Minimum numeric value validator
max (int)	CHECK (col <= value)	Maximum numeric value validator
min_length	No DB effect	Minimum string length validator
max_length	VARCHAR(value) column type	Maximum string length validator

pattern	No DB effect	Regex pattern validator
---------	--------------	-------------------------

Required and Unique

required and unique are top-level field keys, not rule types. This avoids ambiguity — required: true is unambiguous YAML. unique: true generates both a UNIQUE INDEX and a pre-insert uniqueness check in the service layer.

3.3.2 Relations

Relations declare foreign key relationships. The tool generates the FK column, index, and join query helpers. It does not generate ORM-style lazy loading — all joins are explicit query functions.

```
relations:
  - has_many:   Order           # this resource has many Orders
    fk:         user_id         # FK column name on the Order table
    cascade:    true            # (opt) bool - ON DELETE CASCADE, default: false

  - has_one:    Profile         # this resource has one Profile
    fk:         user_id

  - belongs_to: User            # this resource has a user_id FK
    fk:         user_id
    nullable:   false           # (opt) bool - whether FK can be null

  - many_to_many: Tag           # generates join table automatically
    through:    user_tags       # (opt) str - custom join table name
```

Tool generates for has_many: GetOrdersByUserID query, FK column on Order table, index on FK column. Tool generates for belongs_to: FK column on this table, index, GetUser query helper on this resource.

3.3.3 Endpoints

Endpoints declare which HTTP operations are exposed for this resource. The tool generates the full handler, route registration, auth middleware, request/response DTOs, and error mapping for each.

```
endpoints:
  - op:         create          # (req)
    enum[create,get,list,update,delete,custom]
    method:     POST            # (opt) enum[GET,POST,PUT,PATCH,DELETE] -
    inferred from op
    path:       /users          # (opt) str - inferred from resource name if
    absent
    auth:       public          # (req) enum[public, jwt, apikey, hmac]
```



```

    roles:      [admin]                # (opt) list<str> - role names from auth.roles
    owner:      true                    # (opt) bool - service checks resource.user_id ==
caller.id
    paginate:   true                    # (opt) bool|enum[cursor,offset] - for list ops
    name:       CheckoutOrder          # (opt) str - required when op: custom

# — Inbound webhook endpoint (auth: hmac) —————
# Outbound webhooks = external HTTP calls, use the externals block.
# Inbound webhooks = API endpoints with HMAC signature verification.
- op:          custom
  name:         StripeWebhook
  method:       POST
  path:         /webhooks/stripe
  auth:         hmac                  # HMAC signature verification middleware
  hmac_secret:  ${STRIPE_WEBHOOK_SECRET} # (req for hmac) env var
  hmac_header:  Stripe-Signature       # (req for hmac) header to read
  hmac_format:  stripe                 # (req for hmac) enum[stripe, github, raw]
  # Tool generates: endpoint + HMAC verification + 401 on bad sig.
  # Developer implements handler body in hooks/ as a custom endpoint hook.

```

Webhook Design

Outbound webhooks are external HTTP calls — declare them in the externals block with retry and signing. Inbound webhooks are just API endpoints with auth: hmac instead of auth: jwt. No separate webhook DSL block is needed.

op value	Default method	What tool generates
create	POST	Handler + DTO + 201 response + validation
get	GET	Handler + 404 on not found + auth check
list	GET	Handler + pagination wrapper + filter params
update	PUT	Handler + partial update DTO + owner check if set
delete	DELETE	Handler + 204 response + soft delete if configured
custom	any	Route + auth shell only — handler body is your hook

Auth Types

auth value	What tool generates
public	No auth middleware — endpoint is open
jwt	JWT validation middleware, extracts caller ID and role into context
apikey	API key header validation middleware (X-API-Key)
hmac	HMAC-SHA256 signature verification — used for inbound webhooks from 3rd parties

3.3.4 Query DSL

Queries declare every DB operation needed beyond basic CRUD. The tool generates type-safe repository functions for each and automatically derives the required indexes.

```
queries:
# --- Lookups ---
- find_by: [user_id]                # GetOrdersByUserID(ctx, userID)
- find_by: [user_id, status]        # GetOrdersByUserIDAndStatus(ctx, userID,
status)
- find_by: [email]                  # GetUserByEmail(ctx, email) - :one

# --- Range queries ---
- find_by: [created_at]
  op:      between                  # GetOrdersCreatedBetween(ctx, from, to)
- find_by: [total]
  op:      gte                      # enum[eq,neq,gt,gte,lt,lte,between,like,in]

# --- Existence ---
- exists: [email]                   # UserExistsByEmail(ctx, email) bool
- exists: [user_id, status]         # OrderExistsByUserIDAndStatus(ctx, ...) bool

# --- Aggregates ---
- count: [user_id]                  # CountOrdersByUserID(ctx, userID) int64
- sum:   [total, user_id]           # SumTotalByUserID(ctx, userID) decimal
- avg:   [total, status]            # AvgTotalByStatus(ctx, status) decimal
- min:   [total, user_id]           # MinTotalByUserID(ctx, userID) decimal
- max:   [total, user_id]           # MaxTotalByUserID(ctx, userID) decimal

# --- Pagination ---
- paginate: cursor                  # enum[cursor, offset]
  order_by: [created_at: desc]      # list<field: direction>
  default_limit: 20                  # (opt) int - default: 20
  max_limit:   100                  # (opt) int - default: 100

# --- Soft delete ---
- soft_delete: true                 # adds deleted_at column, filters everywhere

# --- Bulk operations ---
- bulk_create: true                 # BatchCreateOrders(ctx, []Order)
- bulk_update: [status]             # BatchUpdateOrderStatus(ctx, ids, status)
- bulk_delete: true                 # BatchDeleteOrders(ctx, ids)

# --- Ordering (global default for list queries) ---
- order_by: [created_at: desc, total: asc]

# --- Custom SQL - tool wraps in type-safe function ---
- custom: GetOrdersWithItemCount
  returns: many                     # enum[one, many]
  sql: |
    SELECT o.*, COUNT(i.id) as item_count
    FROM orders o
    LEFT JOIN order_items i ON i.order_id = o.id
    WHERE o.user_id = $1
    GROUP BY o.id
```

Auto Index Derivation

The tool reads every `find_by`, `exists`, `count`, `sum`, `avg`, `min`, `max`, and `order_by` declaration and generates the minimum set of covering indexes. You never need to manually declare indexes — they are derived from what you actually query.

3.3.5 Rules Block

Rules declare what happens at lifecycle events. Actions fall into two categories: fully generated (the tool writes all code) or hook stub (the tool writes the function signature and call site, the developer implements the body in their hooks file).

```
rules:
  on_create:
    - hash: password          # GENERATED: bcrypt hash before insert
    - check_unique: email     # GENERATED: lookup + ErrEmailTaken
    - compute: total from items # HOOK STUB: developer implements calculation
    - validate: age_check     # HOOK STUB: developer implements
  ValidateAgeCheck
    - guard: owner_or_admin   # GENERATED: caller is owner or has admin role
    - after:
      - emit: OrderCreated    # GENERATED: runs after successful DB write
      - send_email: welcome   # GENERATED: publish to message broker
      - call: InventoryService.Reserve # GENERATED: inter-service call stub

  on_update:
    - guard: owner_or_admin   # GENERATED: auth guard
    - guard: status_transition # GENERATED: state machine transition check
    - compute: recalc_total   # HOOK STUB
    - after:
      - emit: OrderUpdated

  on_delete:
    - soft: true              # GENERATED: sets deleted_at, never hard
  deletes
    - guard: owner_or_admin
    - after:
      - emit: OrderDeleted

  on_read:
    - guard: owner_or_admin   # GENERATED: auth guard on reads
```

Rule Action Reference

Action	Type	What tool generates
hash: field	GENERATED	bcrypt.GenerateFromPassword call
check_unique: field	GENERATED	Repo lookup before insert + named error
guard: owner_or_admin	GENERATED	Caller ID vs resource.user_id + role check

guard: status_transition	GENERATED	Full state machine guard from states block
soft: true	GENERATED	deleted_at field + soft delete query + list filter
emit: EventName	GENERATED	Typed producer.Publish call
send_email: template	GENERATED	mailer.Send(ctx, template, data) call
call: Svc.Method	GENERATED	Typed inter-service call + error propagation
compute: name	HOOK STUB	Hook function signature + call site with nil check
validate: name	HOOK STUB	Validator hook signature + call after struct validation

3.3.6 State Machine

Declares valid state transitions for a resource. The tool generates: the transition guard function (called from guard: status_transition), domain error ErrInvalidTransition, and an OnStateChange hook called after every successful transition.

```
states:
  field:      status          # (req) str - the enum field that holds state
  transitions:          # (req) list<Transition>
    - from: pending    to: confirmed
    - from: confirmed  to: shipped
    - from: shipped     to: delivered
    - from: pending    to: cancelled
    - from: confirmed  to: cancelled
  # The tool generates ErrInvalidTransition for all other from→to combinations.
  # OnStateChange hook is called after every successful transition.
```

3.3.7 Hook Declarations

Standard lifecycle hooks (BeforeCreate, AfterCreate, etc.) are always generated. Use the hooks block to declare additional custom hook points that the tool will add to the generated Hooks struct.

```
hooks:
  # Standard hooks - always generated, no need to declare:
  # BeforeCreate, AfterCreate, BeforeUpdate, AfterUpdate,
  # BeforeDelete, AfterDelete, ValidateCreate, ValidateUpdate,
  # OnStateChange (when states block present)

  # Custom hook points - declare extras here:
  custom:
    - name: BeforePasswordReset # (req) str - PascalCase
      input:                    # (opt) list<Param>
        - name: email    type: str
```

```

        output: error                                # (opt) enum[error, (type, error)] – default:
error

    - name: AfterEmailVerified
      input:
        - name: user      type: User
      output: error

```

3.3.8 Where Business Logic Lives

This section explains the complete separation between generated structural code and developer-written business logic. Understanding this is essential for working with GoForge.

The Core Rule

Generated code lives in `generated/` and is read-only. Business logic lives in `hooks/` and is owned entirely by the developer. The two never share a file. The tool only calls into developer code through the generated hook interface.

What the tool generates for a resource (example: User)

```

generated/user/
  model.go           ← User struct, UserStatus enum, CreateUserRequest,
UpdateUserRequest DTOs
  repository.go      ← CreateUser, GetUserByID, GetUserByEmail, UpdateUser,
                        SoftDeleteUser, ListUsers, CountUsers – all type-safe
  queries.sql        ← Raw SQL for all declared queries (sqlc input for Go)
  service.go         ← CreateUser, GetUser, UpdateUser, DeleteUser, ListUsers
                        – structural logic only, hook call sites baked in
  handler.go         ← HTTP handler for each endpoint – parse req, call service, write
resp
  routes.go          ← Route registration with auth middleware applied
  hooks.go           ← UserHooks struct with all hook function fields
  errors.go          ← var ErrEmailTaken, ErrNotFound, ErrForbidden, etc.
  mock.go            ← MockUserRepository and MockUserService for testing

```

What the developer writes (example: User)

```

hooks/user/user.hooks.go

func RegisterUserHooks(svc *generated.UserService) {

    // ValidateCreate – extra business validation after generated format checks
    svc.Hooks.ValidateCreate = func(ctx context.Context, req *CreateUserRequest)
error {
    if isDisposableEmail(req.Email) {
        return ErrDisposableEmail
    }
    if req.Age < 18 {
        return ErrUnderage
    }
    return nil
}
}

```

```

    // BeforeCreate - runs after validation, before DB insert
    svc.Hooks.BeforeCreate = func(ctx context.Context, req *CreateUserRequest)
error {
    // password is already hashed by generated code (hash: password rule)
    // add your own pre-create logic here
    req.ReferralCode = generateReferralCode()
    return nil
}

    // AfterCreate - runs after successful DB insert
    svc.Hooks.AfterCreate = func(ctx context.Context, user *User) error {
    // emit + send_email already handled by generated code (after: rules)
    // add anything else here
    return creditWelcomeBonus(ctx, user.ID)
}

    // OnStateChange - called on every status transition
    svc.Hooks.OnStateChange = func(ctx context.Context, from, to UserStatus, user
*User) error {
    if to == UserStatusSuspended {
        return revokeAllSessions(ctx, user.ID)
    }
    return nil
}
}

```

Execution order inside generated service (CreateUser example)

Step	Source	What runs
1	GENERATED	Struct-level field validation (req, email, min:N etc.)
2	GENERATED	ValidateCreate hook called (your extra business rules)
3	GENERATED	Uniqueness check on email (generated repo lookup)
4	GENERATED	Password bcrypt hash (from hash: password rule)
5	HOOK	BeforeCreate hook called (your pre-insert logic)
6	GENERATED	DB insert via generated repository
7	HOOK	AfterCreate hook called (your post-insert logic)
8	GENERATED	Event emission + email send (from after: rules)
9	GENERATED	Return typed User to handler

3.4 Types Block

Declares shared value objects that can be used as field types across any resource. Types are pure value objects — no ID, no table, no endpoints. They are stored as JSONB inside the parent resource row and generate typed structs with built-in validators.

Types vs Resources

Use types: for value objects that live inside a parent row (Money, Address, GeoPoint). Use resources: for entities that need their own table, ID, and endpoints. A Payment has a Money amount — Money is a type. A Payment is a resource.

```
types:
  - name: Money
    fields:
      - name: amount
        type: decimal
        required: true
      - name: currency
        type: str
        required: true
    rules:
      - type: min_length
        value: 3
      - type: max_length
        value: 3

  - name: Address
    fields:
      - name: street    type: str    required: true
      - name: city      type: str    required: true
      - name: country   type: str    required: true
      - name: postcode  type: str    nullable: true

# Using custom types as field types in any resource:
resources:
  - name: Payment
    fields:
      - name: amount      type: Money    required: true
      - name: billing_addr type: Address nullable: true
```

Concern	Generated output
Go struct	generated/types/money.go — typed struct with json + db tags
Java class	generated/types/Money.java — class with Jackson + Bean Validation
DB column	JSONB — custom types always stored as JSON inside parent row
Validator	Validate() method generated from field rules, called by parent service
Import	Automatically added to every file that uses the type — never manual
private: true	Private fields on custom types excluded from all serialisation

3.3.9 DTOs and Mappers

Every resource generates a fully separated DTO layer. The DB model struct is never used as an API request or response. The tool generates per-operation request DTOs, per-operation response DTOs, event payload DTOs, and all mapper functions between them. No mapper code is written by hand.

Private field guarantee

Fields marked `private: true` are excluded from the DTO system at generator level — not by convention. The validator enforces this: if a DTO definition includes a private field, parsing fails with an error before any code is generated. `password_hash` leaking to an API response is structurally impossible.

DTO kinds

kind	Used for	What tool generates
<code>request</code>	API input	Struct with validation tags. No DB tags. Password field in plain text.
<code>response</code>	API output	Struct with JSON tags only. Computed field hook stubs.
<code>event</code>	Kafka payload	Struct with JSON tags. Versioned. Stable schema separate from API.
<code>internal</code>	Repo params	Typed params struct. No tags. Never serialised externally.

DTO DSL syntax

```
resources:
  - name: User
    fields: [...]

  dtos:
    - name: CreateUserRequest
      kind: request
      op: create
      fields:
        - first_name
        - last_name
        - email
        - name: password      # override field — different name/type from model
          type: str
          required: true
          rules:
            - type: min_length
              value: 8

    - name: UpdateUserRequest
      kind: request
      op: update
```



```

    fields:
      - name: first_name
        optional: true # PATCH semantics - all fields optional
      - name: email
        optional: true
        immutable: true # validator rejects direct email changes

- name: UserResponse
  kind: response
  ops: [get, update] # one DTO can serve multiple endpoint ops
  fields:
    - first_name
    - last_name
    - email
    - role
    - name: full_name
      type: str
      compute: true # generates ComputeFullName hook stub

- name: UserSummaryResponse
  kind: response
  ops: [list]
  fields:
    - first_name
    - last_name
    - role
    # email deliberately excluded from list response

- name: UserCreatedEvent
  kind: event
  fields:
    - first_name
    - last_name
    - email
    - role

```

Generated mapper layer

The tool generates `mapper.go` (Go) or `UserMapper.java` (Java) with every conversion function. These are the only places where DB model to DTO conversion happens. Handler uses mapper. Service uses mapper. Nobody else does.

```

// generated/user/mapper.go - fully generated, never edit

func FromCreateRequest(req CreateUserRequest) User {
    return User{ FirstName: req.FirstName, LastName: req.LastName, Email: req.Email
}
    // Password excluded - BeforeCreate hook hashes it separately
}

func ToUserResponse(u User) UserResponse {
    return UserResponse{
        ID: u.ID, FirstName: u.FirstName, LastName: u.LastName,
        Email: u.Email, Role: string(u.Role), Status: string(u.Status),
        // FullName set by ComputeFullName hook after this returns
    }
}

```

```

func ToUserSummaryResponse(u User) UserSummaryResponse {
    return UserSummaryResponse{
        FirstName: u.FirstName, LastName: u.LastName,
        Role: string(u.Role), Status: string(u.Status),
    }
}

func ToUserCreatedEvent(u User) UserCreatedEvent {
    return UserCreatedEvent{
        FirstName: u.FirstName, LastName: u.LastName,
        Email: u.Email, Role: string(u.Role),
    }
}

```

3.5 Transactions Block

Declares multi-resource atomic DB operations. The user defines what SQL to run — the tool generates all the transaction boilerplate: BEGIN, COMMIT, ROLLBACK, error propagation, and connection management. No transaction code is ever written by hand.

Design Principle

Transaction orchestration code is 100% standard and fully deterministic from the declared steps. The user only declares intent (what SQL to run, in what order). The tool generates everything around it.

Local Transaction — same DB, fully atomic

All steps run inside a single DB transaction. If any step fails, all previous steps are rolled back automatically.

```

transactions:
  - name: PlaceOrder                # (req) str - PascalCase, becomes generated
    function name
    type: local                     # (req) enum[local] - saga added in Phase 2
    steps:                          # (req) list<TransactionStep>
      - name: CreateOrder           # (req) str - step label for error messages
        sql: |                     # (req for sql step) raw SQL, $N params
          INSERT INTO orders (user_id, total, status)
          VALUES ($1, $2, 'pending')
          RETURNING id
        params:                    # (req) list<Param> - maps to $1, $2 ...
          - name: user_id
            type: int
          - name: total
            type: decimal

      - name: DecrementInventory
        sql: |
          UPDATE inventory
          SET quantity = quantity - $1
          WHERE product_id = $2 AND quantity >= $1

```

```

    params:
      - name: quantity    type: int
      - name: product_id  type: int
      error_if_rows: 0      # (opt) int - fail tx if affected rows == N
      error: InsufficientStock # (opt) str - domain error name to raise

- name: CreatePaymentRecord
  sql: |
    INSERT INTO payments (order_id, amount, status)
    VALUES ($1, $2, 'pending')
  params:
    - name: order_id  type: int
    - name: amount    type: decimal

after:                                # (opt) list<Action> - runs AFTER commit succeeds
- emit: OrderPlaced
- send_email: order_confirmation

```

Tool generates for local transactions: a single typed function `PlaceOrder(ctx, params)` that opens a DB transaction, executes each SQL step in order with the tx handle, commits on success, rolls back on any error, and executes after actions post-commit.

What the tool generates (Go example)

```

// generated - do not edit
func (s *OrderService) PlaceOrder(ctx context.Context, req PlaceOrderRequest) error {
    tx, err := s.db.BeginTx(ctx, nil)
    if err != nil { return fmt.Errorf("begin tx: %w", err) }
    defer tx.Rollback()

    // Step 1: CreateOrder
    var orderID int64
    err = tx.QueryRowContext(ctx,
        `INSERT INTO orders (user_id, total, status) VALUES ($1, $2, 'pending')
RETURNING id`,
        req.UserID, req.Total,
    ).Scan(&orderID)
    if err != nil { return fmt.Errorf("CreateOrder: %w", err) }

    // Step 2: DecrementInventory
    res, err := tx.ExecContext(ctx,
        `UPDATE inventory SET quantity = quantity - $1 WHERE product_id = $2 AND
quantity >= $1`,
        req.Quantity, req.ProductID,
    )
    if err != nil { return fmt.Errorf("DecrementInventory: %w", err) }
    if rows, _ := res.RowsAffected(); rows == 0 {
        return ErrInsufficientStock
    }

    // Step 3: CreatePaymentRecord
    _, err = tx.ExecContext(ctx,
        `INSERT INTO payments (order_id, amount, status) VALUES ($1, $2,
'pending')`,
        orderID, req.Total,
    )
    if err != nil { return fmt.Errorf("CreatePaymentRecord: %w", err) }
}

```

```

    if err := tx.Commit(); err != nil { return fmt.Errorf("commit: %w", err) }

    // After actions (post-commit)
    s.events.Publish(ctx, OrderPlacedEvent{OrderID: orderID})
    s.mailer.Send(ctx, "order_confirmation", req.UserID)
    return nil
}

```

Saga — Phase 2

Sagas handle distributed transactions across multiple services where a single DB transaction is not possible. The user declares the action and compensation hook for each step. The tool generates the full saga orchestrator: step sequencer, state tracking, and compensation chain on failure.

```

# Phase 2 — not in v1
transactions:
  - name: PlaceOrderSaga
    type: saga
    steps:
      - name: ReserveInventory
        action: OnReserveInventory      # hook you implement
        compensate: OnReleaseInventory  # hook called on rollback

      - name: ChargePayment
        action: OnChargePayment
        compensate: OnRefundPayment

      - name: ConfirmOrder
        action: OnConfirmOrder
        compensate: OnCancelOrder
    # Tool generates: orchestrator that runs steps in order,
    # calls compensate hooks in reverse on any failure.

```

3.6 Auth Block

Declares the full authentication subsystem. The tool generates login, logout, token refresh, and password reset as complete endpoints with no hook required for the structural flow. Hook points exist for customising behaviour at key moments.

```

auth:
  provider:      jwt                # (req) enum[jwt, apikey, none]
  secret:        ${JWT_SECRET}      # (req for jwt) str — env var reference
  expiry:        24h                 # (opt) duration — default: 24h
  refresh_expiry: 7d                 # (opt) duration — default: 7d

  login:
    via:          [email, password] # (req) list — identifies login method
    on_success:    [emit:UserLoggedIn] # (opt) list<Action>
    on_failure:    [emit>LoginFailed]  # (opt) list<Action>

```

```

    rate_limit:    5/min          # (opt) str - requests per time window
    hook:          OnLogin        # (opt) str - custom hook after successful login

logout:
  invalidate:     refresh_token  # (opt) enum[refresh_token, all_sessions]

refresh_token:
  storage:        redis          # (opt) enum[redis, db] - where tokens stored

password_reset:
  via:            email          # (req) enum[email, sms]
  expiry:         1h             # (opt) duration
  hook:           OnPasswordReset # (opt) str - hook after reset validated

roles:            [admin, user, guest] # (opt) list<str> - valid role names

permissions:
  - role: admin   can: [read,write,delete] on: all
  - role: user    can: [read,write]        on: own
  - role: guest   can: [read]              on: public

```

3.7 Messaging Block

```

messaging:
  broker:    kafka          # (req) enum[kafka, rabbitmq, sqs]
  brokers:   [{KAFKA_BROKER_1}] # (req) list<str> - broker addresses

  producers:
    - topic:    order.created # (req) str
      event:    OrderCreated  # (req) str - matches emit: value in rules
      key:      order_id      # (opt) str - partition key field name
      schema:   # (opt) explicit schema; inferred from event
name if absent
  - order_id:   int
  - user_id:    int
  - total:      decimal
  - created_at: timestamp

  consumers:
    - topic:    payment.completed # (req) str
      event:    PaymentCompleted  # (req) str - generates typed event struct
      group:    orders-service    # (req) str - consumer group ID
      handler:  OnPaymentCompleted # (req) str - hook name developer implements
      retry:
        attempts: 5          # (opt) int - default: 3
        backoff:  exponential # (opt) enum[fixed, linear, exponential]
        dlq:      payment.failed.dlq # (opt) str - dead letter topic
      concurrency: 4          # (opt) int - parallel consumer goroutines

```

Tool generates for producers: typed Publish function, schema validation, key extraction. Tool generates for consumers: listener loop, deserialisation, retry logic, DLQ routing, and the typed handler hook interface. Developer implements only the handler hook body.

3.8 Externals Block

Declares all outbound HTTP dependencies — both third-party APIs and other internal services. Inter-service calls are treated identically to external API calls. The consuming service declares a typed client interface; the tool generates the HTTP client that makes the call. gRPC and other transports are Phase 2.

Inter-Service Call Pattern

When order-service calls user-service, declare UserServiceClient in the externals block. The tool generates a typed client with GetUser(), SuspendUser() etc. as HTTP calls. The URL comes from an env var (USER_SERVICE_URL). No service registry or discovery in Phase 1 — intentionally simple.

```
externals:
  # — Internal service client (inter-service call) —————
  - name:      UserServiceClient
    type:      http
    base_url:   ${USER_SERVICE_URL}    # URL from env var — simple, works everywhere
    auth:       bearer_token           # pass caller JWT through to downstream
    timeout:    5s
    retry:
      attempts: 3
      backoff:   exponential
    calls:
      - name:      GetUser
        method:     GET
        path:       /users/:id
        response:   UserResponse

      - name:      SuspendUser
        method:     POST
        path:       /users/:id/suspend
        body:       SuspendUserRequest
        response:   void

  # — Third-party external API —————
  - name:      StripeClient
    type:      http
    base_url:   ${STRIPE_BASE_URL}
    auth:       bearer_token
    auth_header: Authorization
    timeout:    10s
    retry:
      attempts: 3
      backoff:   exponential
      on_status: [429, 502, 503]
    headers:
      Stripe-Version: "2023-10-16"
    calls:
      - name:      ChargeCard
        method:     POST
        path:       /v1/charges
        body:       ChargeRequest
        response:   ChargeResponse
        errors:
          - status: 402
```

```

        error: CardDeclined
      - status: 429
        error: RateLimited

# — Outbound webhook delivery —————
# Outbound webhooks = external HTTP POST with retry + HMAC signing.
- name:      WebhookDelivery
  type:      http
  base_url:  ${WEBHOOK_SUBSCRIBER_URL}
  auth:      hmac
  hmac_secret: ${WEBHOOK_SIGNING_SECRET}
  hmac_header: X-Signature
  timeout:   10s
  retry:
    attempts: 5
    backoff:  exponential
  calls:
    - name:    DeliverEvent
      method:  POST
      path:    /
      body:    WebhookPayload
      response: void

```

Tool generates for all externals: typed client struct, all method implementations with request/response serialisation, timeout/retry wrapper, HTTP status to domain error mapping, HMAC signing if configured, and a mock client for testing. The env var for each base_url is automatically added to the generated Config struct and startup validator.

3.9 Jobs Block

```

jobs:
  - name:      ExpireOrders          # (req) str - PascalCase
    schedule:  "0 * * * *"          # (req) str - standard cron expression
    description: Cancel stale pending orders # (opt) str - docs only
    timeout:   5m                    # (opt) duration - job killed if exceeded
    lock:      true                  # (opt) bool - distributed lock, default: true
    lock_ttl:  10m                   # (opt) duration - lock expiry

    # Option A: pure SQL job - fully generated, no hook needed
    db_query: |
      UPDATE orders SET status = 'cancelled'
      WHERE status = 'pending'
      AND created_at < NOW() - INTERVAL '24 hours'
    after: [emit:OrdersExpired]      # (opt) list<Action> - after successful query

    # Option B: hook job - tool generates scheduler + wrapper, you write logic
    hook: GenerateDailyReport        # (opt) str - hook name you implement

    # Option C: inter-service call - fully generated
    call: InventoryService.SyncAll   # (opt) str - Service.Method

    retry:
      attempts: 2                    # (opt) int - job-level retry on failure
      backoff:  fixed

```

3.10 Cache Block

```
cache:
  provider: redis                # (req) enum[redis, memcached]
  url:      ${REDIS_URL}         # (req) str
  prefix:   orders-svc          # (opt) str - namespace all keys, default:
project name

# Per-query cache config (inside resource queries block):
queries:
  - find_by: [id]
    cache:
      ttl:          5m           # (req) duration
      key:          "user:{id}"  # (req) str - template with field refs
      invalidate_on: [update, delete] # (opt) list<enum[create,update,delete]>
```

3.11 Storage Block

```
storage:
  provider: s3                   # (req) enum[s3, gcs, azure_blob, local]
  bucket:   ${S3_BUCKET}        # (req) str
  region:   us-east-1           # (opt) str

# Per-resource upload config (inside resource block):
uploads:
  - name:      UploadAvatar      # (req) str - generates POST /users/:id/avatar
    field:     avatar_url        # (req) str - resource field updated after
upload
  allowed_types: [image/jpeg, image/png, image/webp] # (req) list<str>
  max_size:     5mb              # (req) str
  path:         "avatars/{user_id}/{timestamp}"      # (req) str - S3 key template
  transform:    # (opt) image transforms (tool generates
pipeline)
    - resize: 256x256
    - format: webp
  after: [update:avatar_url]     # (opt) list<Action> - after successful upload
```

3.12 Observability Block

```
observability:
  metrics: prometheus           # (opt) enum[prometheus, datadog, none]
  tracing: jaeger               # (opt) enum[jaeger, zipkin, otel, none]
  logging: structured           # (opt) enum[structured, plain] - default:
structured

  auto:                         # (opt) auto-instrument these on every endpoint
    - request_duration          # histogram: http_request_duration_seconds
    - request_count             # counter:    http_requests_total
    - error_rate                # gauge:     http_error_rate
    - db_query_duration         # histogram: db_query_duration_seconds

  custom_metrics:               # (opt) declare custom Prometheus metrics
```



```

- name:      orders_revenue_total
  type:      counter          # enum[counter, gauge, histogram, summary]
  help:      "Total revenue from completed orders"
  labels:    [currency, status]
  emit_on:   AfterCreateOrder # str - hook name that triggers this metric

```

3.13 Middleware Block

```

middleware:          # Applied globally to all routes in order
- logging            # Structured request/response logging
- recovery           # Panic recovery → 500 response
- cors               # CORS headers (permissive by default)
- rate_limit: 100rpm # str - Nrpm | Nrps | N/min | N/hour
- custom: TenantResolver # str - references your extension middleware

```

3.14 Extensions Block

Extensions are full subsystems the developer writes. The tool mounts them into the router and injects the Dependencies struct. Use extensions for anything the DSL does not cover:

WebSocket servers, gRPC gateways, SOAP adapters, custom protocols.

```

extensions:
- name: RealtimeNotifications # (req) str - must match extensions/{name}/ dir
  path: extensions/realtime/  # (opt) str - default: extensions/{lowercase
name}/
  mounts:                    # (req) list<Mount>
  - route: /ws/notifications
    middleware: [jwt]        # (opt) list<str> - standard middleware applied

- name: LegacySOAPAdapter
  mounts:
  - route: /legacy
    middleware: []

# Developer implements in extensions/realtime/extension.go:
# type RealtimeNotifications struct{}
# func (e *RealtimeNotifications) Mount(g *gin.RouterGroup, deps *Dependencies) {
#   // full control - deps gives you DB, Redis, Kafka, Config, all services
# }

```

3.15 Overrides Block

Overrides change the default behaviour of generated code globally. Use when the DSL defaults do not match your conventions.

```

overrides:
  error_format: custom # enum[default, custom]
                  # custom: tool calls your FormatError(err) hook

```

```
    response_wrapper:  custom          # enum[default, custom]
                                   # custom: tool calls your WrapResponse(data)
hook
  middleware_order:          # Override the default middleware execution order
    - cors
    - rate_limit
    - auth
    - logging
    - custom: TenantResolver
  pagination_style:  envelope          # enum[envelope, headers] - default: envelope
                                   # envelope: {data:[], meta:{total, cursor}}
                                   # headers: Link, X-Total-Count headers
    id_type:              uuid          # enum[int, uuid] - default: int (BIGSERIAL)
    timestamps:          true          # bool - add created_at/updated_at to all,
default: true
  soft_delete_global: false          # bool - apply soft delete to all resources,
default: false
```

4. Code Generation Architecture

4.1 File Ownership Model

The central principle of GoForge's architecture is strict file ownership. Every file is owned by exactly one party — the tool or the developer — and that ownership never changes.

Directory	Owner	Behaviour on regenerate
generated/	Tool	Wiped and fully rewritten every time
hooks/	Developer	Never read, never modified by tool
extensions/	Developer	Never read, never modified by tool
custom/	Developer	Never read, never modified by tool

Immutability Enforcement

All files inside generated/ are set to chmod 444 (read-only) after generation. The filesystem prevents accidental edits. On regeneration the tool temporarily lifts permissions, rewrites, and re-locks.

4.2 Generated Directory Structure

generated/	
{resource}/	
model.go	← struct, types, enums, request/response DTOs
repository.go	← all DB operations (sqlc-backed for Go)
queries.sql	← raw SQL for all declared queries
service.go	← structural service with hook call sites baked in
handler.go	← HTTP handler for each declared endpoint
routes.go	← route registration with auth middleware
hooks.go	← UserHooks struct with all function pointer fields
errors.go	← domain error vars (ErrNotFound, ErrEmailTaken, etc.)
mock.go	← MockUserRepository + MockUserService
messaging/	
producers.go	← typed Publish functions for all declared producers
consumers.go	← listener loops with hook call sites
externals/	
{client}.go	← typed HTTP client with retry + error mapping
{client}_mock.go	← mock client implementing same interface
jobs/	
scheduler.go	← cron wiring for all declared jobs
{job}.go	← job wrapper with timeout + distributed lock
auth/	
middleware.go	← JWT validation, role checks, permission guards
handler.go	← login, logout, refresh, password reset endpoints
migration/	
{N}_{name}.sql	← versioned migration files, never modified after creation
config.go	← typed Config struct + startup validator
wire.go	← full DI wiring – all constructors called in correct order
routes.go	← top-level router with all routes + extension mount points

`.goforge.lock` ← JSON snapshot of spec at last generation

4.3 Import Management

All import statements in generated files are managed automatically. The developer never writes an import in any generated file. Imports in hook files are the developer's responsibility — those are the only files they own.

Import Collector

Before rendering each template, the generator runs an Import Collector that walks the resolved resource and builds the exact import set needed. It checks every field type, every rule, every hook, every external dependency, and every relation to determine which packages are required.

```
// Import Collector logic — simplified
func CollectServiceImports(resource ResolvedResource) ImportSet {
    set := ImportSet{}
    set.Add("context")           // always needed
    set.Add("fmt")              // always needed

    for _, field := range resource.Fields {
        switch field.Type {
        case TypeTimestamp: set.Add("time")
        case TypeUUID:      set.Add("github.com/google/uuid")
        case TypeJSON:      set.Add("encoding/json")
        case TypeCustom:    set.Add(module + "/generated/types")
        }
    }
    for _, rule := range resource.Rules {
        switch rule.Type {
        case RuleHash: set.Add("golang.org/x/crypto/bcrypt")
        case RuleEmit: set.Add(module + "/generated/messaging")
        }
    }
    return set
}
```

goimports — the safety net

After template rendering, every Go file is processed by goimports. This adds any import the Collector missed, removes any unused import, and groups imports in stdlib / internal / external order. Generated Go files always compile — a missing import is impossible.

Java import resolution

Java has no goimports equivalent, so the Java Import Collector is more exhaustive. Every field type, every validation annotation, every Spring/Jackson/framework import is mapped explicitly.

The Collector is the source of truth — if it does not map a type, the validator catches it before generation.

Import hierarchy — circular dependency prevention

The Resolver enforces a strict one-way import DAG. No generated file can import from a layer above it. Violations are caught at resolve time before any file is written.

Layer	May only import from
generated/types/	Standard library only
generated/{r}/model.go	types/ only
generated/{r}/dto.go	types/ only
generated/{r}/repository.go	model.go
generated/{r}/mapper.go	model.go + dto.go
generated/{r}/service.go	repo + mapper + types + externals + messaging
generated/{r}/handler.go	service + dto only
generated/wire.go	all services — always generated last

go.mod management

The GoMod generator maintains go.mod automatically. It derives all required external packages from the same Import Collector sets used by all other generators. On goforge update, if new dependencies are needed, go.mod is updated and go mod tidy runs automatically. Developers only add to go.mod for their own hook dependencies.

4.4 Update & Regeneration Contract

Command	Behaviour
goforge generate	First run — generate all files from spec
goforge update	Diff spec vs lock, rewrite generated/ only, skip hooks/
goforge diff	Print what would change without writing any files
goforge validate	Parse and validate spec, report all errors, no file output

goforge add resource <n>	Add new resource files without touching existing ones
goforge hooks scaffold <r>	Generate empty hooks file for a resource in hooks/

5. Escape Hatches

GoForge is a platform, not a prison. Three levels of escape are available when the DSL does not cover a use case.

Level	Name	When to use	What you write
1	Hook	Business logic inside existing flow	Hook function body only
2	Custom endpoint	New route on existing resource, no gen logic	Entire handler in hooks/
3	Extension	Entirely new subsystem outside DSL model	Everything inside extensions/

Extensions receive the Dependencies struct giving access to: DB connection, Redis client, Kafka producer, Config, Logger, Metrics registry, and every generated service. Extensions are not starting from scratch — they inherit all generated infrastructure.

6. Design Decisions

The following questions were raised during spec design and have been resolved. Each decision is recorded here with rationale so future contributors understand why the DSL is shaped the way it is.

Question	Decision	Rationale
Transactions / Sagas	Local tx block in DSL (Phase 1). Saga DSL (Phase 2).	Transaction code is 100% standard — fully generatable. User declares SQL steps, tool owns BEGIN/COMMIT/ROLLBACK.
Webhooks	No separate block. Outbound = externals. Inbound = custom endpoint with auth: hmac.	Webhooks are just HTTP with specific auth. Folding into existing patterns keeps DSL surface small.
WebSockets / SSE	Level 3 extension in Phase 1. SSE DSL block considered for Phase 2.	Too stateful for DSL in Phase 1. PubSub added to Dependencies so extensions can use it.
Multi-tenancy	Not a platform concern. Removed from scope entirely.	Platform-level tenancy breaks cross-tenant use cases. Users add tenant_id as a normal field with their own queries.
DSL syntax style	Structured sub-objects throughout. No terse string codes.	Unambiguous YAML — no string splitting. IDE schema validation works. Token count is not the bottleneck in Phase 1.
Mono-repo support	One goforge.yaml per service. Workspace support is a later phase.	Phase 1 targets single service / monolith. Most users start there. Mono-repo added when the tool is validated.
Inter-service calls	Treated as externals block. HTTP only in Phase 1. gRPC added in Phase 2.	Same pattern as third-party APIs — typed client, URL from env var, retry, mock. No service registry complexity in Phase 1.

7. Build Roadmap

Phase 1 — Core MVP

Goal: generate a running, production-grade CRUD service from a spec in under 5 minutes.

- DSL parser and schema validator with full error reporting and line numbers
- Structured field definitions — types, required, unique, nullable, rules
- Relations — has_many, has_one, belongs_to, many_to_many
- Migration SQL generation with versioning and lock file
- Model / struct + enum + DTO generation
- Repository generation — all CRUD + declared queries (sqlc-backed for Go)
- Auto-index derivation from query declarations
- Service generation with hook call sites baked in
- Handler + routes generation
- Hook interface generation + goforge hooks scaffold command
- Error types generation
- DI wiring (wire.go) generation
- Local transaction block — BEGIN / COMMIT / ROLLBACK fully generated
- Auth subsystem — JWT, roles, permissions, login, logout, refresh, password reset
- Externals block — typed HTTP clients for third-party APIs, inter-service calls, and outbound webhooks
- Inbound webhooks — custom endpoint with auth: hmac
- State machine generation
- Soft delete support
- Cursor and offset pagination
- goforge generate, update, diff, validate, add commands

Phase 2 — Integrations & Real-Time

Goal: cover all common backend integration patterns.

- Messaging DSL — Kafka, RabbitMQ, SQS producers and consumers with DLQ
- Saga DSL — distributed transaction orchestrator with compensation hooks
- Scheduled jobs with distributed locking (cron + db_query + hook variants)
- Object storage (S3, GCS) with upload endpoints and image transforms
- Cache integration — Redis read-through + cache invalidation

- SSE DSL block — one-way server-sent event streams
- WebSocket support — Level 3 extension with PubSub in Dependencies
- gRPC transport for externals and inter-service calls
- Java Spring Boot parity with Go output

Phase 3 — Platform & Observability

Goal: production-grade observability, extensions, and global customisation.

- Observability — Prometheus metrics, Jaeger tracing, structured logging auto-instrumented on every route and DB call
- Custom endpoint generation (op: custom with full auth shell)
- Extension system — Level 3 with Dependencies injection
- Global overrides — error format, response wrapper, middleware order, id type, timestamps
- Storage block — S3/GCS upload endpoints

Phase 4 — Ecosystem

Goal: make GoForge the standard layer between AI and backend code.

- VS Code extension — spec syntax highlighting, validation, hover docs, field autocomplete
- AI prompt kit — system prompts for Claude Code / Cursor to output valid GoForge DSL
- Plugin API — community-contributed generators for new frameworks and languages
- Template registry — shareable resource specs (User with auth, SaaS subscription, etc.)
- Mono-repo workspace support — shared types, multi-service generation

8. Appendix — Complete Annotated DSL Example

A full `goforge.yaml` for a two-resource orders service demonstrating every major DSL feature. This is the reference example for AI agents authoring specs.

```
version: 1
project: orders-service
lang:    go
framework: gin
db:      postgres

# — Config —
config:
  - name:    DATABASE_URL
    type:    str
    required: true
  - name:    REDIS_URL
    type:    str
    required: true
  - name:    JWT_SECRET
    type:    str
    required: true
    min_length: 32
  - name:    STRIPE_URL
    type:    str
    required: true
  - name:    USER_SERVICE_URL
    type:    str
    required: true
  - name:    STRIPE_WEBHOOK_SECRET
    type:    str
    required: true
  - name:    PORT
    type:    int
    default: 8080
  - name:    LOG_LEVEL
    type:    enum
    values:  [debug, info, warn, error]
    default: info

# — Auth —
auth:
  provider: jwt
  expiry:   24h
  login:
    via:    [email, password]
    rate_limit: 5/min
  refresh_token:
    storage: redis
  password_reset:
    via:    email
    expiry: 1h
  roles: [admin, user]
  permissions:
    - role: admin
      can: [read, write, delete]
      on:  all
    - role: user
      can: [read, write]
      on:  own
```

```
# — Resources
resources:

  - name: User
    fields:
      - name: name
        type: str
        required: true
        rules:
          - type: min_length
            value: 2
          - type: max_length
            value: 50

      - name: email
        type: str
        required: true
        unique: true
        rules:
          - type: email

      - name: password
        type: str
        required: true
        private: true
        rules:
          - type: min_length
            value: 8

      - name: role
        type: enum
        values: [admin, user]
        default: user

      - name: status
        type: enum
        values: [active, suspended]
        default: active

      - name: phone
        type: str
        nullable: true

    relations:
      - has_many: Order
        fk: user_id

    endpoints:
      - op: create
        path: /users
        auth: public

      - op: get
        path: /users/:id
        auth: jwt

      - op: list
        path: /users
        auth: jwt
```

```
    roles:    [admin]
    paginate: offset

  - op:    update
    path:   /users/:id
    auth:   jwt
    owner:  true

  - op:    delete
    path:   /users/:id
    auth:   jwt
    roles:  [admin]

  - op:    custom
    name:   ResendVerification
    method: POST
    path:   /users/:id/resent-verification
    auth:   jwt
    owner:  true

# Inbound webhook from an identity provider
- op:    custom
  name:   IdentityProviderWebhook
  method: POST
  path:   /webhooks/identity
  auth:   hmac
  hmac_secret: ${IDP_WEBHOOK_SECRET}
  hmac_header: X-Signature-256
  hmac_format: raw

queries:
  - find_by: [email]
  - find_by: [status]
  - count:
      field: role
  - soft_delete: true
  - paginate: offset
  order_by:
    - field: created_at
      direction: desc

rules:
  on_create:
    - type: hash
      field: password
    - type: check_unique
      field: email
    - type: validate
      name: age_check
    - type: after
      actions:
        - type: emit
          event: UserCreated
        - type: send_email
          template: welcome
  on_update:
    - type: guard
      name: owner_or_admin
  on_delete:
    - type: soft
    - type: after
      actions:
```

```
        - type: emit
          event: UserDeleted

states:
  field: status
  transitions:
    - from: active
      to:   suspended
    - from: suspended
      to:   active

hooks:
  custom:
    - name:   AfterEmailVerified
      input:
        - name: user
          type: User
      output: error

errors: [EmailTaken, Underage, DisposableEmail]

- name: Order
  fields:
    - name:   user_id
      type:   int
      required: true

    - name:   status
      type:   enum
      values: [pending, confirmed, shipped, delivered, cancelled]
      default: pending

    - name:   total
      type:   decimal
      required: true
      rules:
        - type: min
          value: 0

    - name: items
      type: json

    - name:   notes
      type:   str
      nullable: true

  relations:
    - belongs_to: User
      fk:         user_id

  endpoints:
    - op:   create
      path: /orders
      auth: jwt

    - op:   get
      path: /orders/:id
      auth: jwt
      owner: true
```

```

- op:      list
  path:    /orders
  auth:    jwt
  owner:   true
  paginate: cursor

- op:      update
  path:    /orders/:id
  auth:    jwt
  owner:   true

- op:      custom
  name:    CancelOrder
  method:  POST
  path:    /orders/:id/cancel
  auth:    jwt
  owner:   true

queries:
- find_by: [user_id, status]
- sum:
  field:    total
  group_by: user_id
- count:
  field:    user_id
- paginate: cursor
  order_by:
    - field: created_at
      direction: desc
  default_limit: 20
- soft_delete: true
- custom: GetOrdersWithItemCount
  returns: many
  sql: |
    SELECT o.*, COUNT(i.id) as item_count FROM orders o
    LEFT JOIN order_items i ON i.order_id = o.id
    WHERE o.user_id = $1 GROUP BY o.id

rules:
  on_create:
    - type: compute
      name: total
      from: items
    - type: after
      actions:
        - type: emit
          event: OrderCreated
  on_update:
    - type: guard
      name: status_transition
    - type: after
      actions:
        - type: emit
          event: OrderUpdated
  on_delete:
    - type: soft

states:
  field: status
  transitions:
    - from: pending
      to: confirmed

```

```

    - from: confirmed
      to: shipped
    - from: shipped
      to: delivered
    - from: pending
      to: cancelled
    - from: confirmed
      to: cancelled

    errors: [InvalidTransition, EmptyOrder, PaymentFailed]

# — Transactions —————
transactions:
  - name: PlaceOrder
    type: local
    steps:
      - name: CreateOrder
        sql: |
          INSERT INTO orders (user_id, total, status)
          VALUES ($1, $2, 'pending')
          RETURNING id
        params:
          - name: user_id
            type: int
          - name: total
            type: decimal

      - name: DecrementInventory
        sql: |
          UPDATE inventory
          SET quantity = quantity - $1
          WHERE product_id = $2 AND quantity >= $1
        params:
          - name: quantity
            type: int
          - name: product_id
            type: int
        error_if_rows: 0
        error: InsufficientStock

      - name: CreatePaymentRecord
        sql: |
          INSERT INTO payments (order_id, amount, status)
          VALUES ($1, $2, 'pending')
        params:
          - name: order_id
            type: int
          - name: amount
            type: decimal

    after:
      - type: emit
        event: OrderPlaced
      - type: send_email
        template: order_confirmation

# — Externals —————
externals:

  # Inter-service call — user-service treated same as any external
  - name: UserServiceClient

```



```

    type:      http
    base_url:  ${USER_SERVICE_URL}
    auth:      bearer_token
    timeout:   5s
    retry:
      attempts: 3
      backoff:  exponential
    calls:
      - name:      GetUser
        method:    GET
        path:       /users/:id
        response:   UserResponse

      - name:      SuspendUser
        method:    POST
        path:       /users/:id/suspend
        body:       SuspendUserRequest
        response:   void

# Third-party payment API
- name:      StripeClient
  type:      http
  base_url:  ${STRIPE_URL}
  auth:      bearer_token
  timeout:   10s
  retry:
    attempts: 3
    backoff:  exponential
    on_status: [429, 502, 503]
  headers:
    Stripe-Version: "2023-10-16"
  calls:
    - name:      ChargeCard
      method:    POST
      path:       /v1/charges
      body:       ChargeRequest
      response:   ChargeResponse
      errors:
        - status: 402
          error:  CardDeclined
        - status: 429
          error:  RateLimited

    - name:      RefundCharge
      method:    POST
      path:       /v1/refunds
      body:       RefundRequest
      response:   RefundResponse

# — Jobs —————
jobs:
  - name:      ExpireOrders
    schedule:  "0 * * * *"
    timeout:   5m
    db_query: |
      UPDATE orders SET status = 'cancelled'
      WHERE status = 'pending'
      AND created_at < NOW() - INTERVAL '24 hours'
    after:
      - type:  emit
        event: OrdersExpired

```

```
- name:      DailyRevenueReport
  schedule: "0 9 * * *"
  timeout:   10m
  hook:      GenerateDailyReport
```

```
# — Cache —————
cache:
  provider: redis
  url:      ${REDIS_URL}
  prefix:   orders-svc
```

```
# — Observability —————
observability:
  metrics: prometheus
  tracing: jaeger
  logging: structured
  auto:
    - request_duration
    - request_count
    - error_rate
    - db_query_duration
```

```
# — Middleware —————
middleware:
  - logging
  - recovery
  - cors
  - rate_limit: 100rpm
```

```
# — Overrides —————
overrides:
  id_type:   uuid
  timestamps: true
```